16 EXCEPTION HANDLING • DETAILED SOLUTIONS

programming solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Programming Questions (Module 16: Exception Handling)

Solution to Q9 (`student_marks` Range Validation)

CPP

```cpp
#include <iostream>
using namespace std;

class student_marks {
private:
    int marks;

public:
    void input(int m) {
        if (m < 0) {
            throw -1;
        }
        if (m > 100) {
            throw 1;
        }
        marks = m;
    }

    void display() const {
        cout << "Valid Marks: " << marks << endl;
    }
};

int main() {
    student_marks s;

    try {
        s.input(85);
        s.display();
        s.input(120); // Throws +1
    }
    catch (int code) {
        if (code == -1) {
            cout << "Error: Marks cannot be negative!" << endl;
        } else if (code == 1) {
            cout << "Error: Marks cannot exceed 100!" << endl;
        }
    }

    return 0;
}
/*
Expected Output:
Valid Marks: 85
Error: Marks cannot exceed 100!
*/
```

Solution to Q10 (`safe_array` with Custom `index_error`)

CPP

```cpp
#include <iostream>
using namespace std;

class index_error {
private:
    int bad_idx;

public:
    index_error(int idx) {
        bad_idx = idx;
    }

    void show() const {
        cout << "Index out of bounds: " << bad_idx << endl;
    }
};

class safe_array {
private:
    int arr[5] = {10, 20, 30, 40, 50};

public:
    int get(int index) const {
        if (index < 0 || index >= 5) {
            throw index_error(index);
        }
        return arr[index];
    }
};

int main() {
    safe_array sa;

    try {
        cout << "Element at 2: " << sa.get(2) << endl;
        cout << "Element at 7: " << sa.get(7) << endl;
    }
    catch (const index_error& err) {
        err.show();
    }

    return 0;
}
/*
Expected Output:
Element at 2: 30
Index out of bounds: 7
*/
```